UNIVERSITY OF CAMBRIDGE

MPhil in Data Intensive Science – Coursework Assignment

Due on Wednesday 16th December at 23:59

M1 – Machine Learning for Data Intensive Science
Dr Priscilla Canizares

Attempt all questions. The coursework has four steps, worth 15, 25, 35 and 25 marks, 100 marks in total; the marks for each question are given in the right margin in square brackets.
Write a report of not more than 3,000 words, figure captions and tables included. The limit is an upper limit; a shorter report can earn full marks.
Lay the report out with one section per step, titled "Step 1" to "Step 4", and in each section one numbered answer per question, numbered as in this paper ("2.3" is step 2, question 3).
Hand in your work in your private GitLab project at `https://gitlab.developers.cam.ac.uk/phy/data-intensive-science-mphil/assessments/m1_coursework/<YourCRSid>`*. Only you have access to this project, and your access expires after the deadline, 23:59 on 16th December 2026. The project must contain the following files, which Section 1 describes:*

- *A* `README.md` *file which describes exactly how to run your scripts.*
- *The report as a PDF file saved in the location* `report/report.pdf`*.*
- *The results table* `results_table.xlsx`*, the scripts* `step1.py` *to* `step4.py` *with a* `requirements.txt`*, the figures they save in* `figures/`*, the three CSV files in* `outputs/` *and the check file written by* `check_submission.py`*.*

Comment your code so that a reader can follow what it does.
The paper contains 11 sides including this one.

© 2026 University of Cambridge

1 What to hand in

Hand in, in your private GitLab project (its address is on the front page), the report, the results table, the code, the three CSV files and the check file of the self-check below. Lay the project out as follows:

File or folder	What it holds
`README.md`	How to run your scripts
`report/report.pdf`	The report
`results_table.xlsx`	The results table
`step1.py` to `step4.py`	The four scripts and your other `.py` files
`requirements.txt`	The packages you used and their versions
`check_submission.py`	The self-check script, as handed out
`check_ab123.txt`	The check file, with your CRSid in place of `ab123`
`figures/`	The figures of your report
`outputs/`	The three CSV files
`data/`	The two data files, as handed out

The report. One PDF, saved as `report/report.pdf`, of not more than 3,000 words, figure captions and tables included; a shorter report can earn full marks:

- One section per step, titled "Step 1" to "Step 4".
- In each section one numbered answer per question, in order, numbered as in Section 4 (e.g., "2.3" is step 2, question 3); write"not attempted" under a question you did not do.
- Every figure and table captioned with the number of the question it belongs to, and referred to in the text.

The results table. The file `results_table.xlsx`, handed out with the data, has one row for each value the questions ask for, keyed to the question numbers. Fill in its Value column, one value per cell in the form its Format column says, with three significant figures unless the question asks otherwise. Write "not attempted" in the rows of a question you did not do and "not obtained" (with the reason in your answer) where you did it but got no value; leave no cell empty. Keep the file's name and its rows: the self-check below reads it and reports every row that is missing or malformed.

The code. One Python script per step, `step1.py` to `step4.py`, each running from the top folder of your GitLab project as `python3 step1.py` without errors, printing its rows of the results table, one line per row in the form `row 2.1a:  1`, and saving the figures and files of your report; functions may live in other `.py` files, each with a docstring. Save every figure of your report in a folder `figures/`, named by the question it belongs to: `figures/2.2.png`, or `2.2a.png` and `2.2b.png` when a question has several. Say in a comment at the top of each script how long it runs. Include a `requirements.txt` with

© 2026 University of Cambridge

the packages you used and their versions, and a `README.md` file which describes exactly how to run your scripts: the Python version, the command that installs the packages and the command that runs each script. Hand in scripts, not notebooks.

The self-check. Put `step1.py` to `step4.py`, your other `.py` files, `results_table.xlsx` and `check_submission.py` (handed out with the data) in the top folder of your GitLab project. In that folder run, with the Python you use for your scripts and your CRSid in place of `ab123`,

```
python check_submission.py ab123
```

It runs the four scripts one after the other, compares the rows they print with your `results_table.xlsx`, checks that each script sets `SEED = 0`, checks the three CSV files, and writes `check_ab123.txt`. Upload the whole folder to your GitLab project: the report, the scripts, the table, the outputs of the scripts and `check_ab123.txt`. If you change any file after the run, run the check again.

© 2026 University of Cambridge

The seed. Write `SEED = 0` at the top of each script and use `SEED` for every random number generator: the `random_state` of the network, of k-means, of the Gaussian mixtures and of t-SNE, and the NumPy generator of your own code. Running a script again must give the numbers of your report. Your results for the network, the clusters and the maps can still differ a little from everyone else's, since they also depend on your code, on the versions of the libraries and on your machine.

The three CSV files. Header line `object_id,predicted_class`, one row per object, the labels spelled as in the learning set (`active payload`, `dead payload`, `rocket body`, `fragment`). An algorithm checks these files: keep the names and the spellings exactly. Save the three files in the folder `outputs/`.

- `test_predictions_linear.csv`: your linear classifier's prediction for every test object (step 2).
- `test_predictions_network.csv`: your network's prediction for every test object (step 3).
- `suspect_labels.csv`: every training or validation object whose label you believe is wrong, with the label you believe correct (step 3).

2 Coursework introduction

You are given two data sets corresponding to simulated measurements of real satellites and space debris. Each object has one of four labels (classes):

- *Active payload* (`PAY`): a working satellite that fires its engine now and then to climb back to its altitude (a re-boost).
- *Dead payload* (`PAY` as well): a satellite that is not functional.

- *Rocket body* (`R/B`): the upper part of a rocket, left in orbit after a launch.
- *Fragment* (`DEB`): a piece of an object that broke up. A *breakup* is an explosion or a collision that shatters one object into many fragments, each on an orbit close to the parent's.

The space debris is on a low Earth orbit, 400 to 900 km above the ground, where an object circles the Earth in about an hour and a half.

Dataset 1. The learning set (`learning_set.npz`) follows 3,463 objects for 128 days, with three values per object per day:

- The semi-major axis a, in km: the size of the orbit, measured from the centre of the Earth (the height above the ground is a minus the Earth's radius, 6378 km).
- The eccentricity e: the shape of the orbit.
- The drag force term: how strongly the thin upper atmosphere slows the object down.

The orbits are not in one plane: the inclination and the other angles of each orbit are in the catalogue (Table 1), not in the learning set. The file splits the objects into training, validation and test sets and gives a label for the training and validation objects. For the 695 test objects the label is *withheld*: removed from the file, which shows -1 instead. Your classifiers have to predict it.

Dataset 2. The catalogue (`catalogue.csv`). Space-surveillance networks follow objects in orbit with radar and telescopes and publish a short description of each object's orbit (called an *element set*) about once a day. The catalogue imitates such a record for 284 other objects over one year. Each row gives the time, the six *orbital elements* that describe the orbit at that time, the drag term and the label (Table 1). The labels are coarser than in the learning set: active and dead payloads share the label `PAY`, rocket bodies are labelled `R/B` and fragments `DEB`, and for 85 objects the label is withheld (the column is empty).

You do not need orbital mechanics for this coursework. Your task is to analyse the data in the four steps of Section 4. Each step uses only the provided data and can be solved independently. Listing 1 reads the two data files. The learning set is one array of shape (objects, 3, days), hence an `.npz` file; the catalogue has a different number of rows per object, hence a `.csv` table. As in real catalogues, a few labels are wrong and a few measurements are far from the orbit they belong to. The three textbooks of the course [1–3] cover every method you need; the other references are the sources of the physics in Sec. 3, and you do not need to read them.

3 Orbital evolution

Two effects change the orbit of the objects over days and months, and the data were simulated with these two alone:

© 2026 University of Cambridge

Table 1: Columns of the catalogue, one row per element set.

Column	What it is	Unit
`object_id`	The number that identifies the object	–
`epoch_day`	The time of the element set, in days from the start of the year	day
`a_km`	Semi-major axis a: the size of the orbit, measured from the centre of the Earth (height above the ground: $a - 6378$ km)	km
`e`	Eccentricity e: the shape of the orbit, 0 for a circle; a few thousandths here, so every orbit is close to a circle	–
`i_deg`	Inclination i: the tilt of the orbit relative to the equator; 0° above the equator, 90° over the poles, above 90° against the Earth's rotation	degree
`raan_deg`	Longitude of the ascending node Ω: where the orbit crosses the equator going north, as an angle along the equator from a fixed direction in space; with i it fixes which way the orbit faces	degree
`argp_deg`	Argument of perigee ω: where along the orbit the marks closest to the Earth lies	degree
`M_deg`	Mean anomaly M: where the object is along its orbit; it changes too fast to follow from one row to the next	degree
`drag_term`	How strongly the thin upper atmosphere slows the object down: large for light objects with a large surface, such as fragments, small for heavy, compact ones; a rough estimate that scatters from row to row	$\mathrm{m^2\,kg^{-1}}$
`object_type`	The label: `PAY` (payload, active or dead), `R/B` (rocket body) or `DEB` (debris: a fragment); empty where withheld	–

© 2026 University of Cambridge

The Earth's bulge. The Earth's radius at the equator is about 21 km larger than at the poles. The extra mass around the equator makes the plane of every tilted orbit turn slowly about the Earth's axis, like a wobbling spinning top: a, e and i stay the same, and Ω changes at the steady rate [4, 5]

$$\dot{\Omega} = -\frac{3}{2}\, n J_2 \left(\frac{R_\oplus}{p}\right)^2 \cos i, \tag{1}$$

where $n = \sqrt{\mu/a^3}$ is the mean motion, the angle in radians per second that the object covers along its orbit; $p = a(1 - e^2)$; $\mu = 3.986004418 \times 10^5\ \mathrm{km^3\,s^{-2}}$ is the gravitational constant times the mass of the Earth; $R_\oplus = 6378.137$ km is the Earth's radius at the equator, and $J_2 = 1.08263 \times 10^{-3}$ measures the size of the bulge. Ω decreases for orbits tilted by less than 90° and increases above 90°, the rate is up to a few degrees per day, fastest for low orbits near the equator and zero over the poles.

The atmosphere. There is a trace of air even 400–900 km above the ground. It takes a little energy from the object on every orbit, so the orbit shrinks at the rate [5]

$$\dot{a} = -\rho(a - R_\oplus)\, \beta \sqrt{\mu a}, \tag{2}$$

Listing 1: Reading the two data files.

```
import numpy as np
import pandas as pd

data   = np.load("data/learning_set.npz")
series = data["series"]   # (objects, 3, days): a [km], e, drag term [m^2/kg]
label  = data["label"]    # index into data["class_names"]; -1 where withheld
split  = data["split"]    # index into data["split_names"]: train/validation/test

cat = pd.read_csv("data/catalogue.csv")    # one row per element set
for object_id, rows in cat.groupby("object_id"):
    rows = rows.sort_values("epoch_day")
    t, a = rows["epoch_day"].to_numpy(), rows["a_km"].to_numpy()
```

where $\rho(h)$ is the density of the air at height h, which falls by about a factor of ten for every 140 km, and β is the drag term, in $\mathrm{m}^2\,\mathrm{kg}^{-1}$: how easily the object is slowed down. Use SI units in Eq. (2). We call $\dot{a}$ the decay rate, and it is largest for light objects with a large surface, such as fragments, and for low orbits, where the air is denser. The drag term in the data is a noisy estimate of β.

Active payloads. When a has dropped by about 1 km, a working satellite fires its engine and climbs back: the re-boost. Its $a(t)$ is a sawtooth, a slow fall then a sudden jump, a dead payload or a rocket body only falls.

© 2026 University of Cambridge

4 Tasks:

Step 1. Explore the learning set [15 marks]

In this part you explore the learning set: which features discriminate the classes, and which labels in the file are inconsistent with the physics of Sec. 3 (label noise).

1. Count the objects in each split (training, validation, test), and the class frequencies in the training and validation sets. Then build a majority-class baseline: a classifier that predicts for every object the class that is most frequent in the training set. Evaluate it on the validation set with two metrics: [3]
   - The accuracy: the fraction of validation objects whose predicted label is the label in the file.
   - The balanced accuracy: the fraction of correctly predicted objects within each class, averaged over the four classes.

   Report both numbers. Use the balanced accuracy as the metric in steps 2 and 3.
2. Plot a, e and the drag term β against the day for two or three objects of each class, chosen by you, one panel per quantity. Then engineer four scalar features from each object's series: [5]

- The mean of a over the 128 days: the altitude.
- The decay rate: the slope of a straight line fitted by least squares to a against the day, in metres per day.
- The number of re-boosts: the days on which a rises by more than a threshold; choose the threshold from your plots.
- The mean of the drag term.

Which classes does each feature help to separate, and why (Sec. 3)? Steps 3 and 4 use these features again.

3. Some labels in the training and validation sets are wrong, and can be considered noise. For each feature, plot one histogram per class on the same axes. An object whose feature value lies in the range of another class is a candidate for a wrong label, e.g., an active payload with no re-boost, or a fragment with the drag term of a payload. Describe your selection rules, count the candidates, and report the count and its fraction of the labelled objects. [5]
4. Build the input vectors of steps 2 and 3: one row of 3×128 numbers per object, the 128 values of a, then those of e, then those of the drag term. Standardise every column with the mean and standard deviation of the training set. Keep every object and its label as given in steps 2 and 3; the candidates of question 3 are used in step 3, question 4. [2]

© 2026 University of Cambridge

Step 2. A linear classifier [25 marks]

In this step you train a regularised linear classifier, select its regularisation strength on the validation set, interpret its weights, and implement gradient descent for its loss. Use the input vectors of step 1, question 4 (build them as described there if you did not finish step 1).

1. Train a multiclass logistic-regression classifier with an L2 regulariser on the training set. Its loss is Eq. (3) below; C is the inverse regularisation strength, so a small C is a strong regulariser. Use class weights inversely proportional to the class frequencies, since the classes are imbalanced (scikit-learn [6]: `LogisticRegression` with `class_weight="balanced"`). Train one model for each $C = 10^{-4}, \ldots, 10^{2}$, in factors of ten, and report the balanced accuracy on the training set and on the validation set for each C, in one table or one plot. Select the C with the highest validation balanced accuracy and say which it is. [6]
2. The logit of each class is a linear combination of the inputs. Take the 128 weights on $a(t)$ of each class, at your C: [6]
   - Plot them against the day, one line per class, and describe their shape.
   - The linear combination $\sum_t w(t)\, a(t)$ over the 128 days is one number per object, like the features of step 1. Say which feature of step 1 it resembles. Check it: compute it for the training objects and its correlation with that feature.

3. Implement gradient descent for the loss [11]

$$J(W, \boldsymbol{b}) = \frac{1}{N} \sum_{n=1}^{N} s_n \, \mathrm{CE}_n + \frac{\lambda}{2} \|W\|^2, \qquad \lambda = \frac{1}{CN}, \tag{3}$$

$$\mathrm{CE}_n = -\ln \frac{\exp(z_{n,y_n})}{\sum_{k=1}^{4} \exp(z_{n,k})}, \qquad \boldsymbol{z}_n = W^{\mathsf{T}} \boldsymbol{x}_n + \boldsymbol{b},$$

where N is the number of training objects, $\boldsymbol{x}_n$ the input vector of object n and y_n its label in the file, W holds the weights (one column per class) and $\boldsymbol{b}$ the biases, $\boldsymbol{z}_n$ is the vector of the four logits of object n, CE_n its cross-entropy, the negative log of the softmax probability given to its label, $s_n = N/(4N_{y_n})$ with N_{y_n} the number of training objects of the class of object n, and $\|W\|^2$ is the sum of the squared weights (the biases are not regularised). Gradient descent updates the parameters $\boldsymbol{\theta} = (W, \boldsymbol{b})$ as $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \, \nabla J(\boldsymbol{\theta})$, with a fixed learning rate η, from $\boldsymbol{\theta} = 0$; write it yourself, with no library minimiser. It uses only the gradient,

$$\nabla_W J = \frac{1}{N} X^{\mathsf{T}} \big[\boldsymbol{s} \odot (P - Y) \big] + \lambda W, \qquad \nabla_{\boldsymbol{b}} J = \frac{1}{N} \sum_{n=1}^{N} s_n \, (\boldsymbol{p}_n - \boldsymbol{y}_n), \tag{4}$$

where the rows of X are the input vectors, the rows of P the softmax probabilities $\boldsymbol{p}_n$ of each object, the rows of Y the one-hot labels $\boldsymbol{y}_n$, and $\boldsymbol{s}\odot$ multiplies row n by s_n.

- The learning rate. Gradient descent converges only for $\eta < 2/L$, where L is the largest curvature of J: the largest eigenvalue of its Hessian H, the matrix of second derivatives. Use $\eta = 1/L$. Compute L once for each C, before the runs, and from gradients alone: the product of H with a vector $\boldsymbol{v}$ is a finite difference of the gradient, $H\boldsymbol{v} \approx [\nabla J(\boldsymbol{\theta} + \epsilon \boldsymbol{v}) - \nabla J(\boldsymbol{\theta} - \epsilon \boldsymbol{v})]/(2\epsilon)$ with ϵ about 10^{-6}, and power iteration (start from a random $\boldsymbol{v}$ and replace $\boldsymbol{v}$ by $H\boldsymbol{v}/\|H\boldsymbol{v}\|$ a hundred times) makes $\|H\boldsymbol{v}\|$ settle at L. Evaluate it at the minimum of J, $\boldsymbol{\theta}^\star$, which you take from a library solver run to convergence (`LogisticRegression` with the same C and class weights, its weights put into Eq. (3)); $J^\star = J(\boldsymbol{\theta}^\star)$ is the minimum of the loss. Report L at $C = 10^{-2}$ and at your C.
- The runs. At $C = 10^{-2}$ and at your C, run 5,000 iterations with $\eta = 1/L$ and report $J - J^\star$ at the end. Report also the condition number $\kappa = L/\lambda$ for both values of C: the ratio of the largest to the smallest curvature of J, the smallest being λ.
- Above the limit. Run 5,000 iterations with $\eta = 2.5/L$ at your C and report what happens.
- Momentum. Add Nesterov's momentum [7]: the velocity $\boldsymbol{v} \leftarrow \theta \boldsymbol{v} - \eta \, \nabla J(W + \theta \boldsymbol{v})$, then $W \leftarrow W + \boldsymbol{v}$ (the same for $\boldsymbol{b}$), with $\eta = 1/L$ and $\theta = (\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$. Run 5,000 iterations at your C and report $J - J^\star$, against plain gradient descent.

© 2026 University of Cambridge

4. Predict the class of every test object with the model of question 1 at your C. Hand in `test_predictions_linear.csv`: columns `object_id` and `predicted_class`, one row per test object. [2]

Step 3. A neural network [35 marks]

In this step you train a multilayer perceptron with and without weight decay, identify the training and validation labels it misclassifies, which are the candidates for label noise, and predict the test labels. Use the input vectors of step 2.

1. Train a feed-forward network with two hidden layers of 256 and 128 ReLU units and a softmax output layer: scikit-learn's `MLPClassifier` with `hidden_layer_sizes=(256, 128)`, or your own implementation. Optimise with Adam [8], mini-batches of 200, learning rate 10^{-3}, for 2,000 epochs. Train two models from the same initialisation: one without weight decay, one with weight decay `alpha` = 1 (the L2 regularisation strength). Every 10 epochs, record each model's training accuracy and validation balanced accuracy (with `MLPClassifier`, call `partial_fit` once per epoch). Plot both learning curves against the epoch, for both models, in one figure. [7]
2. Which model overfits the label noise? Answer from your learning curves, without any true label: [8]
   - Compare each model's training accuracy at the last epoch with one minus the fraction of candidates of step 1 (if you did not finish step 1: with one minus the fraction of wrong labels you expect, and say what you expect).
   - Find the epoch of each model's highest validation balanced accuracy, and describe how it changes after that epoch.

   Say what the two comparisons show.
3. Compare the validation balanced accuracy of the model with weight decay with that of your linear classifier of step 2 at your C (if you did not finish step 2: scikit-learn's `LogisticRegression` at its default settings on the same inputs). A network's validation metric fluctuates by a few hundredths from one epoch to the next, so use the mean over the last 500 epochs, not the last value. Report both. [5]
4. List the training and validation objects that the model with weight decay misclassifies: their label in the file differs from its prediction. Decide for each whether the label or the model is wrong: plot the features of step 1 of these objects, grouped by their label in the file, beside the same features of all objects with that label (if you did not finish step 1: the number of re-boosts and the mean drag term). Features consistent with the predicted class indicate a wrong label; features consistent with the file, a model error. Hand in `suspect_labels.csv`: one row per object whose label you find is wrong, with its `object_id` and, in `predicted_class`, the label you find correct. [12]
5. Predict the class of every test object with the model with weight decay. Hand in `test_predictions_network.csv`, with the columns of step 2. [3]

© 2026 University of Cambridge

Step 4. Without the labels [25 marks]

In this step you apply unsupervised learning to the learning set, and find how many breakups produced the fragments of the catalogue. The labels are used here only for visualization and to evaluate the results.

1. Centre the 128 values of a of each object on its own mean, so that only the shape of the series remains, and run principal component analysis (PCA) on these centred series. Report the explained variance ratio of the first component, and plot the component against the day. Say which feature of step 1 the first principal component score of an object resembles, and check it by the correlation over the objects (if you did not finish step 1: use the slope of a against the day). [6]
2. Compute two t-SNE embeddings [9] of the objects, for example with scikit-learn's `TSNE` at its default settings: one from the first 10 principal components of the standardised input vectors of step 1, question 4, one from a few standardised features per object (those of step 1, or the mean of a, its slope, the mean drag term and the number of re-boosts). Colour the points by the label in the file and say which embedding separates the classes better. Then quantify it: for each object, the fraction of its 10 nearest neighbours, in the embedding's input space and not in the plane, that share its label; report the mean over the objects for each input. [7]
3. Cluster the objects into four clusters, from each of the two inputs, with k-means and with a Gaussian mixture model. Compare the clusters with the labels by the adjusted Rand index [10] (`adjusted_rand_score`: 1 when every cluster holds one class, about 0 for a random assignment). Then fit Gaussian mixture models with 2 to 12 components and compute the BIC [11] of each; the lowest BIC selects the number of components. Report the Rand indices and the selected number of components, against four. [5]
4. The catalogue objects labelled `DEB` are the fragments of a few breakups (Sec. 2), each scattered from one point in space. Using only these objects, find how many breakups there were and which fragments belong to each: [7]
   - Choose the orbital element that the bulge and the atmosphere leave unchanged, so that it stays constant over time (Sec. 3 says which elements change and which do not): the fragments of one breakup still share the value they had at the breakup. Take its median over the year for each object.
   - Fit Gaussian mixture models with 1 to 5 components to the medians, and select the number of components by the BIC.
   - Two components whose means lie close together are one breakup with a non-Gaussian spread: merge them, and say how you decided. A component that holds one or two objects is not a breakup but a wrong label: leave it out and say so.

   Report the number of breakups and the fragments of each.

© 2026 University of Cambridge

References

[1] Kevin P. Murphy. *Probabilistic Machine Learning: An Introduction*. The MIT Press, 2022.

[2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. The MIT Press, 2016.

[3] Pankaj Mehta, Marin Bukov, Ching-Hao Wang, Alexandre G. R. Day, Clint Richardson, Charles K. Fisher, and David J. Schwab. A high-bias, low-variance introduction to machine learning for physicists. *Phys. Rep.*, 810:1–124, 2019. doi: 10.1016/j.physrep.2019.03.001.

[4] Yoshihide Kozai. The motion of a close earth satellite. *Astron. J.*, 64(1274): 367–377, 1959.

[5] Oliver Montenbruck and Eberhard Gill. *Satellite Orbits: Models, Methods and Applications*. Springer, Berlin, Heidelberg, 2000. doi: 10.1007/978-3-642-58351-3.

[6] Fabian Pedregosa et al. Scikit-learn: machine learning in Python. *J. Mach. Learn. Res.*, 12:2825–2830, 2011.

[7] Yurii Nesterov. *Introductory Lectures on Convex Optimization: A Basic Course*, volume 87 of *Applied Optimization*. Kluwer Academic Publishers, Boston, 2004. doi: 10.1007/978-1-4419-8853-9.

[8] Diederik P. Kingma and Jimmy Ba. Adam: a method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR), San Diego*, 2015. arXiv:1412.6980.

[9] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *J. Mach. Learn. Res.*, 9:2579–2605, 2008.

[10] Lawrence Hubert and Phipps Arabie. Comparing partitions. *J. Classif.*, 2(1): 193–218, 1985. doi: 10.1007/BF01908075.

[11] Gideon Schwarz. Estimating the dimension of a model. *Ann. Stat.*, 6(2):461–464, 1978. doi: 10.1214/aos/1176344136.

© 2026 University of Cambridge